SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

QUESTION 1: ATM Withdrawal Module – Reverse Engineering Black-Box Tests

1. Understanding and decomposition

The ATM withdrawal module accepts a withdrawal request and produces observable outcomes such as approval, rejection messages, cash dispensing, and account-balance changes. Reverse engineering means inferring the rules that govern these outcomes by designing controlled inputs and comparing the observed results with expected results. Since the source code is unavailable, the exact implementation cannot be asserted; however, its major functional units can be inferred.

Input interface: receives account/card information, PIN, and requested withdrawal amount.
Authentication and account checks: validate the PIN and determine whether the account/card is active or blocked.
Amount validation: checks format, permitted minimum and maximum, and whether the amount follows ATM denomination rules if applicable.
Funds and ATM cash checks: compare the request with available account balance and cash currently available in the ATM.
Transaction execution: authorizes the withdrawal, debits the account, and requests cash dispensing.
Error handling and audit: returns a suitable message and records the outcome; failed transactions must not leave an incorrect debit.

2. Equivalence Class Partitioning and Boundary Value Analysis

Equivalence Class Partitioning (ECP) divides the input domain into classes expected to be handled similarly. A representative value from each class reduces redundant tests, while Boundary Value Analysis (BVA) focuses on the values most likely to expose comparison and off-by-one errors.

Equivalence class	Representative input	Expected result
Valid amount	Amount inside permitted range and valid denomination	Proceed if account and ATM conditions pass
Below minimum	Minimum − one permitted unit	Reject with clear message
At minimum	Exactly minimum	Accept if all other conditions pass
At maximum	Exactly maximum	Accept if all other conditions pass
Above maximum	Maximum + one permitted unit	Reject
Insufficient balance	Valid amount greater than available balance	Reject; balance unchanged

Invalid amount	Zero, negative, blank, alphabetic, malformed decimal	Reject safely
Invalid account state	Wrong PIN, blocked card, inactive account	Deny according to security policy
Insufficient ATM cash	Request exceeds cash available in machine	Reject or offer permitted alternative

If the minimum amount is L and the maximum is U, test L−δ, L, L+δ and U−δ, U, U+δ, where δ is the smallest meaningful amount increment. The actual limits must be taken from the system specification rather than invented.

3. Reconstructed test cases

ID	Scenario	Expected outcome and verification
ATM01	Valid amount within limits; sufficient balance and ATM cash	Withdrawal succeeds; correct amount dispensed and balance reduced once
ATM02	Amount immediately below minimum	Rejected; no debit
ATM03	Amount exactly at minimum	Accepted if all rules pass
ATM04	Amount exactly at maximum	Accepted if all rules pass
ATM05	Amount immediately above maximum	Rejected; no debit
ATM06	Amount exceeds available balance	Rejected; no cash dispensed and balance unchanged
ATM07	Zero, negative, blank, or text amount	Validation message; no crash or transaction
ATM08	Incorrect PIN or blocked card	Access denied; security event logged according to policy
ATM09	ATM has insufficient cash	No unauthorized debit; clear response
ATM10	Network failure after debit request	Transaction status reconciled; no lost or duplicate debit
ATM11	Same request submitted twice	Duplicate transaction prevented or safely identified
ATM12	Cash dispenser failure after authorization	Reversal/rollback or documented recovery; account and cash records consistent

4. Evaluation and improved approach

The original strategy may have used ordinary valid and invalid input-output examples, but the scenario says the module is only partially tested. If boundary cases, insufficient-balance combinations, and failure recovery were not tested, the strategy is inadequate. ECP gives systematic representative classes, while BVA targets threshold errors. Additional decision-table cases should combine account status, balance sufficiency, ATM cash availability, and amount validity.

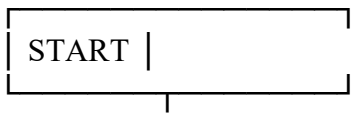
The final test suite should verify more than the displayed message: compare the account ledger before and after, confirm the ATM cash state, verify audit entries, and ensure failed transactions never reduce the balance. Use repeatable regression tests for each defect and supplement structured testing with random malformed-input tests. This approach is more reliable than relying only on a few normal transactions.

QUESTION 2: Execution Traces and Code Snippets – CFG and Path Testing

1. Limitation and conceptual reconstruction

The question refers to execution traces and code snippets, but those artefacts are not reproduced in the supplied task document. Therefore, the following control-flow graph (CFG) is an illustrative reconstruction of a typical validation-and-processing module. The actual decisions, loops, and paths must be verified against the real snippet before exact coverage can be claimed.

2. Conceptual control flow graph



↓
Read input
↓
[Input valid?]
/ \
No Yes
↓ ↓
Show error [Authorised?]
↓ / \
END No Yes
↓ ↓
Deny and log Process request
↓ ↓
END [Success?]
/ \
No Yes

↓↓
Recovery/error Save result
↓↓
END END

3. Decisions, loops, and independent paths

The illustrative graph contains three binary decision points: input validity, authorization, and processing success. If the actual source includes a loop, its condition must have both a loop-back edge and an exit edge. Exception handlers, early returns, and database failures should also appear as separate routes when they affect control flow.

For a single connected graph, cyclomatic complexity can be calculated as $V(G) = E - N + 2$, or as the number of binary decision nodes plus one for a structured graph. This example has three decision nodes, so $V(G) = 3 + 1 = 4$. This indicates four linearly independent basis paths for this simplified graph.

Path 1: invalid input → error message → termination.
Path 2: valid input → authorization denied → log/termination.
Path 3: valid input → authorized → processing failure → recovery/termination.
Path 4: valid input → authorized → processing succeeds → save result/termination.

4. Coverage evaluation and reconstructed strategy

Statement coverage only establishes that statements have executed. A test may execute the validation statement without exercising both outcomes of its decision. Branch coverage requires each decision outcome to be taken. Basis-path testing targets linearly independent routes; full path coverage may be impossible if loops can execute an unlimited number of times.

Construct the CFG directly from source code and validate each edge against execution traces.
Calculate V(G) and design tests for each independent feasible path.
Test each loop with zero iterations, one iteration, multiple iterations, and termination at its limit where relevant.
Include invalid input, exception, database failure, recovery, and successful completion paths.
Use coverage tools to measure statement and branch coverage; report path coverage only for a clearly defined set of feasible paths.

This method improves confidence because it explains which control-flow routes have been exercised and which remain untested. The exact number of paths and coverage percentages cannot be established from the scenario alone; they require the missing code or traces.

QUESTION 3: Web Login and Transaction System – Reconstructing a Hybrid Strategy

1. Infer techniques from defect patterns

Defect logs and test results can reveal likely weaknesses, although they cannot conclusively identify the original test method. Defects immediately below or above limits suggest incomplete Boundary Value Analysis (BVA); malformed values suggest weak Equivalence Class Partitioning (ECP); defects occurring only after a particular sequence suggest missing path or state testing; and mismatches between responses and stored balances suggest integration or transaction-atomicity problems.

2. Decomposition into testable components

Login input validation: empty fields, data formats, password rules, and request size.
Credential verification: correct/incorrect credentials, inactive or locked accounts, and rate limits.
Session and authorization: session creation, expiration, roles, and access restrictions.
Transaction validation: amount, account status, transaction type, and limits.
Transaction processing: balance check, debit/credit, commit, rollback, and retries.
Persistence and audit: database consistency, transaction identifiers, logs, and error handling.

3. Reconstructed test strategy

Component	ECP/BVA tests	White-box or integration paths
Login	Blank, malformed, valid credentials; lockout threshold and adjacent attempt counts	Success, invalid password, locked account, verification service failure, session creation failure
Transaction amount	Negative, zero, minimum, typical, maximum, over-limit amount	Accepted transaction, validation rejection, insufficient funds
Authorization	Valid/expired session; allowed and denied roles	Permitted action and denied action paths
Processing	Duplicate request, timeout, network interruption	Commit, rollback, retry, exception and recovery
Persistence	Existing/missing account, database unavailable	Successful write, failed write, consistency and recovery

4. Gaps and justification

A comprehensive design should combine black-box tests, which check externally visible requirements, with white-box tests, which exercise internal decisions and paths. ECP limits redundancy by choosing representatives from valid and invalid classes. BVA concentrates on numeric thresholds, transaction limits, and lockout boundaries. CFG and basis-path testing cover the internal routes that lead to commit, rollback, or error handling.

The suite should also test concurrent transactions, duplicate submissions, idempotency, access control, audit records, and data integrity. Expected results must include both the user-facing response and the resulting database state. Coverage metrics should be linked to requirements and defect history rather than treated as proof of correctness. Every confirmed production defect should become a regression test.

QUESTION 4: Legacy Inventory System – Inferring Stock Rules from Logs

1. Reconstructing likely business rules

The scenario provides successful and failed transaction logs but not the actual log entries or numeric threshold. Therefore, the exact low-stock rule cannot be confirmed. The likely logic is that stock updates must be validated, inventory must not become negative unless back-ordering is explicitly supported, and a low-stock alert is triggered when the resulting stock reaches a configured threshold.

The process can be decomposed into transaction input, product lookup, quantity validation, stock calculation, database update, low-stock comparison, alert generation, and audit logging. Let S be current stock, Q be the quantity requested for issue, and T be the alert threshold. The resulting stock is $S-Q$ for an issue operation.

2. Equivalence classes and boundaries

Class or boundary	Example condition	Expected result
Invalid quantity	$Q = 0$ or $Q < 0$	Reject; stock unchanged
Valid quantity below stock	$0 < Q < S$	Update stock; evaluate alert threshold
Quantity equals stock	$Q = S$	Stock becomes zero if policy permits; apply shortage/alert rules
Quantity exceeds stock	$Q > S$	Reject or create back-order according to specification
Above alert threshold	Result = $T + 1$	No alert if the rule is stock ≤ T
At alert threshold	Result = T	Alert if the rule is stock ≤ T
Below threshold	Result = $T - 1$	Alert
Concurrent updates	Two requests issue stock at the same time	Prevent overselling and lost updates
Service failure	Database or alert service unavailable	Consistent transaction and recoverable alert state

3. Using logs to infer the hidden boundary

Compare successful and failed quantities for the same product and starting stock. If a request for Q succeeds but Q+1 fails, that suggests a boundary near available stock. Compare alert logs for resulting stock values just above, equal to, and below a suspected threshold to infer whether the comparison is less-than or less-than-or-equal. Repeated observations are needed because other factors, such as permissions or product status, may explain failure.

4. Improved black-box design

Use ECP to cover invalid, valid, and over-stock quantities; BVA around zero, available stock, and the alert threshold; and decision tables combining transaction type, product status, stock level, and alert status. Verify response messages and persisted inventory values. Include concurrent transactions, database failures, and alert-delivery retries. This reconstruction addresses the likely gaps while clearly separating inferred rules from rules confirmed by actual logs.

QUESTION 5: Compiled Library Module – Resource Spikes and Path Testing

1. Interpreting CPU and memory profiles

A CPU or memory spike during stress testing may correspond to repeated searching, nested loops, large collection processing, repeated database queries, excessive object allocation, or a costly decision branch. A compiled executable and resource profile alone do not uniquely reveal source code. These are hypotheses that should be tested by profiling the application under controlled workloads.

2. Functional decomposition

Request parser: reads search, issue, or return requests.
Input validation: checks identifiers, request size, and required fields.
Search engine: scans or indexes book records.
Availability check: decides whether the requested book can be issued.
Transaction processor: updates issue/return status.
Database layer: reads and writes records.
Response and logging: returns results and records failures.

3. Conceptual CFG

START → Validate request → [Valid?]
No → Return error → END
Yes → Search records (loop)
↓
[Match found?] ─No→ Return not found → END
Yes
↓
[Book available?]
No → Return unavailable → END

Yes → Update issue/return record
↓
Persist and respond → END
The search loop may be resource-intensive when the catalogue is large or the search repeatedly scans records. The availability and validation decisions create alternative paths. The actual CFG must be derived from source, symbols, disassembly, or detailed traces; a conceptual graph cannot confirm the internal structure of an executable.

4. Path testing and resource-focused tests
For a connected CFG, cyclomatic complexity is V(G) = E−N+2, or decisions+1 for a conventional structured graph. Once the actual graph is known, identify independent paths and select tests that execute each one. Include an empty catalogue, a small catalogue, a large catalogue, a match at the first and last positions, no match, unavailable books, duplicate records, malformed requests, and database failures.

Measure CPU time, memory consumption, latency, and allocation growth for each workload.
Test increasing catalogue sizes and increasing numbers of concurrent users.
Check loop termination, repeated scans, memory leaks, and unbounded retries.
Inject database delays or failures and check that resources are released.
Repeat the same workloads after optimization and compare results with the baseline.

5. Expected impact
Path testing identifies logical routes that may be expensive or fail under particular conditions, while stress testing reveals whether the resource use becomes unstable at scale. Together they can expose performance bottlenecks, non-terminating loops, excessive allocation, and error-recovery problems. Stability should be judged using response time, resource limits, error rate, and correctness—not CPU percentage alone.

QUESTION 6: Mobile Payment API – Reconstructing State Transitions
1. State-machine interpretation
The question names Pending, Success, Reversal, and Timeout but does not provide the actual JSON responses. The exact state machine therefore cannot be confirmed. A proposed model should be checked against the payment provider's API contract, especially for late responses and reversal rules.
Initiated → Pending → Success
└→ Timeout → Reconciliation
Success → Reversal (if permitted)
Duplicate callbacks → remain in current terminal state
A payment request may first enter Pending. A valid confirmation can change it to Success, while exceeding a response deadline may change it to Timeout. A timed-out request must be reconciled because a timeout does not always prove that the provider failed to process the payment. A permitted reversal may follow a successful transaction. Duplicate callbacks should not create a second debit or reversal.

2. Reconstructed transition table

Current state	Trigger/event	Expected transition and action
Initiated	Valid payment request	Move to Pending; assign unique transaction ID
Pending	Provider confirms payment	Move to Success; record payment once
Pending	Response deadline expires	Move to Timeout; start reconciliation
Timeout	Late success notification	Reconcile according to contract; avoid duplicate debit
Success	Authorized reversal request	Move to Reversal if policy permits
Reversal	Repeated reversal request	Remain Reversal; respond idempotently
Success	Duplicate success callback	Remain Success; do not charge again
Reversal	Unexpected success callback	Reject or ignore according to documented rules
Pending	Malformed or mismatched response	Reject safely; retain consistent transaction record

3. State-based test suite

Test every valid transition and every important invalid transition.
Test transition pairs and sequences, including delayed and out-of-order callbacks.
Retry requests and callbacks to check idempotency.
Test malformed JSON, missing transaction ID, incorrect amount/currency, and unknown status values.

Verify that API responses, database records, ledger balances, and audit logs agree.
Test recovery after timeout, provider outage, and network reconnection.

The implied technique is State Transition Testing because behaviour depends on the current state and the incoming event. It detects sequence errors that isolated input tests can miss. The state table is complete only when it matches the actual documented API contract and covers all legal transitions plus relevant illegal transitions.

QUESTION 7: Encrypted Communication – Fuzzing and Robust BVA

1. Decomposition and purpose

The module accepts input parameters and returns encrypted outputs. Reverse engineering its validation behaviour means observing how it handles valid inputs, boundary lengths, malformed encodings, and invalid cryptographic parameters. Testing must be performed in an authorized isolated environment.

Input parser: extracts fields and validates their types.

Validation layer: checks required fields, length limits, encodings, and permitted algorithms.

Encryption component: performs the configured cryptographic operation.

Output formatter: returns ciphertext, status, and permitted metadata.

Error handler: reports failures without exposing keys, plaintext, or sensitive internal details.

2. Boundary and fuzz test design

Input field	Tests	Expected result
Message length	Empty, 1 byte, maximum−1, maximum, maximum+1, very large	Accept supported sizes; reject excessive sizes safely
Key/nonce/IV	Too short, valid length, too long	Reject invalid cryptographic parameters
Encoding	Malformed UTF-8, invalid Base64/hex, embedded nulls	Controlled validation error
Required fields	Missing, null, wrong type, duplicate field	Reject safely without crash
Special characters	Quotes, separators, control characters, long repeated strings	Treat as data; no command/query injection
Random malformed payloads	Random bytes, truncated data, unexpected combinations	No crash, memory corruption, or secret leakage

BVA should use the actual documented maximum lengths and permitted formats. Fuzzing extends the test space by generating unexpected byte sequences and combinations. Observe crashes, response codes, memory use, timeouts, output length, and whether secrets appear in logs. Use repeatable seeds or saved inputs so that failures can be reproduced.

3. Security justification

Strict length validation and safe memory management reduce buffer-overflow risk. Validating encodings and field types prevents malformed data from reaching unsafe code paths. Parameterized or structured interfaces and treating user input as data reduce injection risk. Encryption output should also be checked for correct format and errors should not reveal keys or plaintext. Black-box fuzzing is valuable but cannot prove the absence of vulnerabilities; source review, dependency checks, and security analysis are complementary.

QUESTION 8: Insurance Premium Calculator – Branch and Path Coverage

1. Why statement coverage was insufficient

100% statement coverage means every executable statement ran at least once. It does not guarantee that both outcomes of a decision were exercised or that every combination of Boolean conditions was checked. For example, a test with an eligible customer and low risk may execute the premium calculation but miss the rejection route and surcharge route.

The prompt does not provide the actual insurance code. The following is an illustrative rule only: if age is eligible AND the policy is active, calculate a premium; if high risk OR claim history is true, apply a surcharge.

2. Conceptual CFG and independent paths

START → [Age eligible AND policy active?]
No → Reject policy → END
Yes → [High risk OR claim history?]
Yes → Apply surcharge → Calculate → END
No → Standard rate → Calculate → END

This simplified graph has two decision nodes, so V(G)=3 for one connected structured graph. The independent paths are: P1—eligibility fails; P2—eligibility passes and the surcharge condition is true; P3—eligibility passes and the surcharge condition is false.

Test	Age eligible	Policy active	High risk	Claim history	Expected result
T1	No	Yes	No	No	Reject policy
T2	Yes	No	No	No	Reject policy
T3	Yes	Yes	Yes	No	Apply surcharge
T4	Yes	Yes	No	Yes	Apply surcharge
T5	Yes	Yes	No	No	Standard premium

3. Reconstructed strategy and completeness

Branch coverage requires both outcomes of each decision. Because the decisions contain compound conditions, tests should also demonstrate the independent effect of each atomic condition where required. Path testing should cover all feasible independent paths in the defined graph. The request asks for 100% branch and path coverage, but that claim can be made only after the actual code and feasible paths are verified; the illustrative graph is not proof of the real module's coverage.

Add boundary tests for age limits, policy start/expiry dates, coverage amount, and surcharge thresholds.

Use independent expected-value calculations to check the premium, surcharge, tax, and rounding.

Test missing or invalid customer data and policy status.
Use regression tests for every calculation defect and measure branch/path coverage after changes.
The reconstructed approach lowers calculation risk by checking alternate decision outcomes and the combinations that statement coverage can miss. It also verifies numerical correctness rather than relying only on code execution.

QUESTION 9: Cloud File Synchronization – ECP, BVA and Stress Testing

1. Inferred equivalence partitions

Lost-file reports associated with file size, type, and network speed suggest that testing may have focused on common files and stable network conditions. The exact partitions used by the original developers cannot be confirmed without their test suite. A revised model should partition each factor into representative classes and test their interactions.
File size: empty, small, typical, large, maximum supported, and over-limit.
File type: supported, unsupported, extension mismatch, unusual Unicode filename.
Network: fast/stable, slow, intermittent, disconnected, and recovered.
File state: unchanged, modified, renamed, deleted, and conflicting versions.
Transfer outcome: completed, interrupted, retried, checksum mismatch, and duplicate request.

2. Revised test cases

Dimension	Representative tests	Expected result
Size boundaries	0 bytes, 1 byte, chunk size−1, chunk size, chunk size+1, maximum, maximum+1	Correct transfer or clear rejection
File type	Supported, unsupported, extension mismatch, Unicode name	Correct handling; no silent omission
Network	Slow transfer, disconnect, timeout, reconnect	Safe retry/resume; incomplete file not marked complete
Concurrency	Same file modified on two devices	Documented conflict resolution without silent data loss
Integrity	Corrupt chunk, checksum mismatch, duplicate retry	Detect corruption and retry/reject safely
Scale	Large batch and many simultaneous clients	No missing files; resource use remains controlled

3. Stress testing and data-loss prevention

Apply BVA at file-size and chunk boundaries, then stress test large batches, many concurrent users, limited disk space, and repeated network interruptions. Verify checksums, metadata, version history, timestamps, and file counts before and after the test. A file should be reported as synchronized only after the complete data is written and integrity verification succeeds.

The system should use durable queues, idempotent retries, atomic finalization, and explicit conflict handling. Logs should allow an interrupted transfer to be traced and resumed. Combining ECP, BVA, stress testing, and integrity checks is stronger than testing only typical files over a stable connection because it addresses both input boundaries and resource/network failures.

QUESTION 10: E-commerce Search and Filter – Decision Table Testing

1. Reverse engineering the query logic

The actual SQL queries and source code are not included, so exact AND/OR/NOT behaviour cannot be asserted. To infer the logic, hold all other parameters constant while changing one filter and compare the returned dataset. Then test combinations to determine whether the conditions are combined using AND, OR, or exclusion logic.

Decompose the module into query parameter parsing, input validation, filter construction, SQL generation, database execution, sorting, pagination, and result formatting. Example filters include category match, price within limit, stock availability, and exclusion of blocked brands.

2. Illustrative decision table

The following table assumes enabled filters are combined using AND, and a blocked-brand condition excludes matching brands. Actual rules must be revised if observed results indicate different logic.

Rule	Category matches?	Price in range?	In stock?	Blocked brand?	Expected result
R1	Yes	Yes	Yes	No	Include item
R2	No	Yes	Yes	No	Exclude if category is mandatory
R3	Yes	No	Yes	No	Exclude
R4	Yes	Yes	No	No	Exclude if in-stock filter is enabled
R5	Yes	Yes	Yes	Yes	Exclude
R6	No	Yes	Yes	No	Include only if category is optional or OR logic is specified

3. Reconstructed test suite

Test each filter individually to establish its effect on the results.
Test pairs of filters and all filters together to identify AND/OR interactions.
Test empty filters, contradictory conditions, missing parameters, invalid price ranges, and zero-result searches.
Test sorting and pagination so records are not duplicated or omitted between pages.
Test special characters and SQL-injection-like inputs safely; use parameterized SQL.
Compare the actual result set with a manually prepared expected set for each rule.

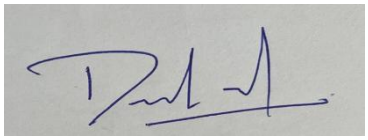
4. Evaluation of completeness

Decision-table testing is complete for a defined rule model when every meaningful rule column has been exercised and the expected output is checked. If there are many values, ECP can select representative values and pairwise testing can reduce redundant combinations, while high-risk interactions are tested explicitly. Completeness cannot be claimed for the unknown original query until actual parameters, SQL logic, and observed datasets are available.

Code of Conduct:

*I **Deepa preya H / 192421047** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation